Key Update for OSCORE (KUDOS)

draft-ietf-core-oscore-key-update-14

Rikard Höglund, RISE
Marco Tiloca, RISE

IETF CoRE WG meeting – IETF 126 – July 23rd, 2026

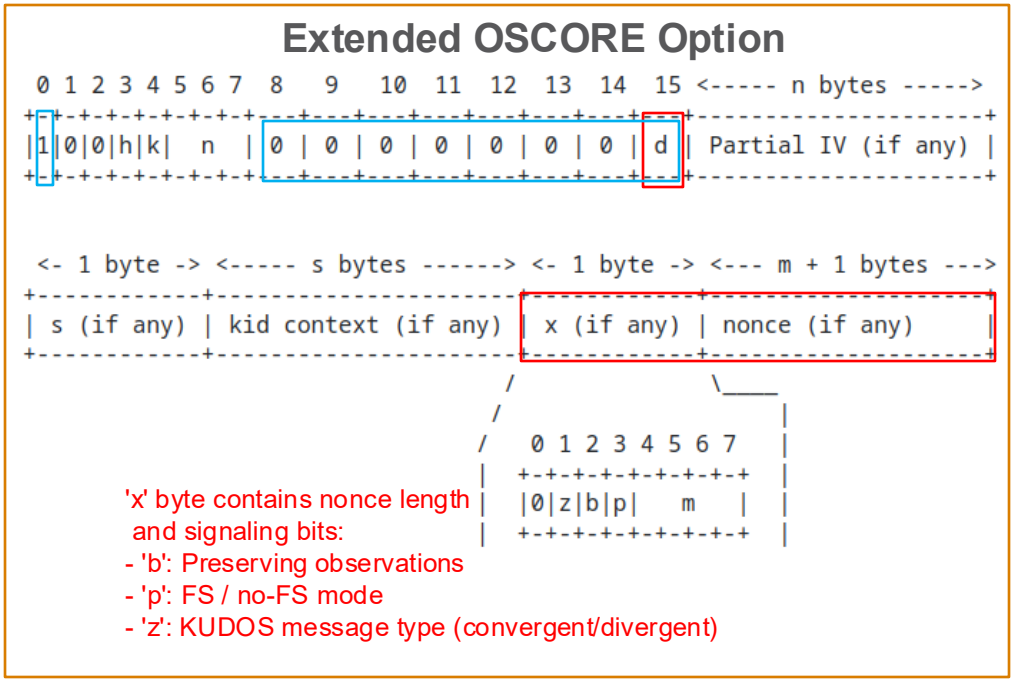
Recap of KUDOS

- **Key Update for OSCORE (KUDOS)**
  - Renew the Master Secret and Master Salt; derive new Sender/Recipient keys
  - No change to the ID Context or Sender/Recipient ID; can achieve Forward Secrecy
  - Agnostic of the key establishment method originally used
  - Loosely inspired by Appendix B.2 of OSCORE
  - The peers update their current context CTX_OLD, deriving a new context CTX_NEW
  - Redesigned in v-10 to use a more flexible and simpler approach

- **Properties**
  - Can be initiated by either peer
  - It is robust against a peer rebooting and loss of state, avoiding the reuse of AEAD (nonce, key)
  - It typically completes in one round trip by exchanging two OSCORE-protected CoAP messages
    - The two peers achieve mutual key confirmation in a following exchange, which is protected with the newly established OSCORE Security Context
  - Flexible in terms of message flow; any CoAP message can be a KUDOS message

Rekeying Procedure

- **Key Update for OSCORE (KUDOS)**
  - Message exchange to share two nonces N1 and N2
    - Decoupled from request/response and client/server concepts
  - Nonces are placed in new fields in OSCORE CoAP option
  - *UpdateCtx()* function for deriving new OSCORE Security Context using the two nonces, two ‘x’ bytes and CTX_OLD
  - Two modes
    - FS mode providing forward secrecy
    - No-FS mode for very constrained devices
  - No change of OSCORE identifiers
  - Expected to complete in 1 round trip

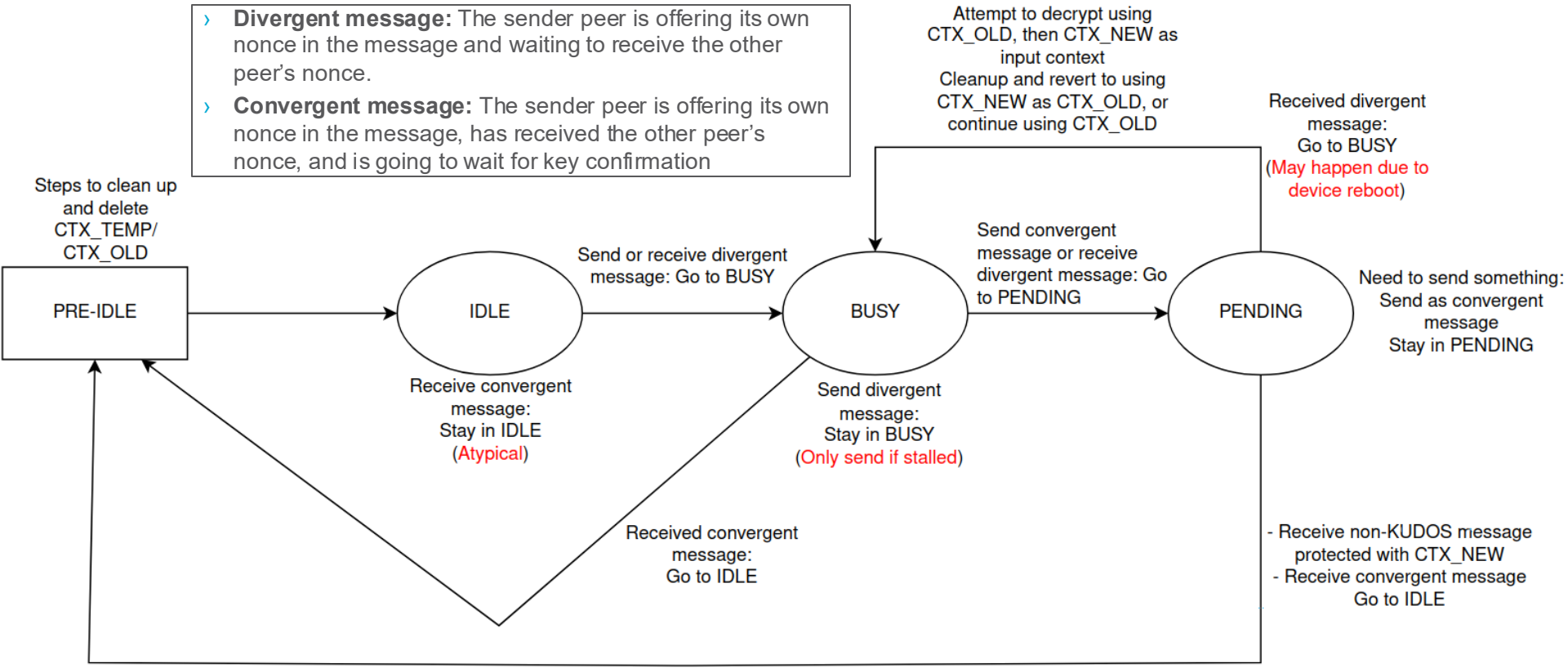


KUDOS States

- **Three possible states: IDLE, BUSY, and PENDING**
  - Normally, the peer is in the IDLE state, i.e., in "equilibrium“
- **A peer starts a KUDOS execution upon entering the BUSY state**
- **A peer successfully completes a KUDOS execution by entering the IDLE state**
  - At which point the peer has the OSCORE Security Context CTX_NEW and has achieved key confirmation
- **A peer can locally represent its current state using 2 bits**
  - (00) IDLE - The peer is not running KUDOS
  - (01) BUSY - The peer has not offered a nonce, but has received the nonce from the other peer
  - (10) BUSY - The peer has offered a nonce, but has not received the nonce from the other peer
  - (11) PENDING: The peer is running KUDOS, has offered its nonce, has received the nonce from the other peer, and is waiting for key confirmation

KUDOS State Machine

- **Divergent message:** The sender peer is offering its own nonce in the message and waiting to receive the other peer's nonce.
- **Convergent message:** The sender peer is offering its own nonce in the message, has received the other peer's nonce, and is going to wait for key confirmation



KUDOS States

- **Three possible states: IDLE, BUSY, and PENDING**
  - Normally, the peer is in the IDLE state, i.e., in "equilibrium“
- **A peer starts a KUDOS execution upon entering the BUSY state**
- **A peer successfully completes a KUDOS execution by entering the IDLE state**
  - At which point the peer has the OSCORE Security Context CTX_NEW and has achieved key confirmation
- **A peer can locally represent its current state using 2 bits**
  - (00) IDLE - The peer is not running KUDOS
  - (01) BUSY - The peer has not offered a nonce, but has received the nonce from the other peer
  - (10) BUSY - The peer has offered a nonce, but has not received the nonce from the other peer
  - (11) PENDING: The peer is running KUDOS, has offered its nonce, has received the nonce from the other peer, and is waiting for key confirmation

Updates for v-14

- **Remove redundant YANG data model for SCHS**
  - Related to a comment to *draft-ietf-schc-8824-update*
  - No need to define the YANG data model in this document

- **Editorial improvements**

- **Updated legal boilerplate and other metadata**

- **Added section "Implementation Status".**
  - Covering 2 working implementations in C and Java
  - These were interoperated during the Hackathon

Implementation Status (1/2)

- **Responsible organization: RISE Research Institutes of Sweden AB**
- **Implementation's name: KUDOS in Californium**
- **Available at: https://github.com/rikard-sics/californium/tree/oscore_kudos**
- **Description: Implementation in Java, based on the Californium CoAP library**
- **Version compatibility: From version -13 onwards.**
- **Licensing: Eclipse Distribution License 1.0 / Eclipse Public License 2.0.**
- **Contact information: Rikard Höglund - rikard.hoglund@ri.se**

Implementation Status (2/2)

- **Responsible organization: University of Murcia (Spain)**
- **Implementation's name: cKUDOS**
- **Available at: https://github.com/francislopezg/cKUDOS**
- **Description: Implementation in C, based on the uOSCORE-uEDHOC library**
- **Version compatibility: From version -13 onwards.**
- **Licensing: Following Apache 2.0 license.**
- **Contact information: Francisco López-Gómez - francisco.lopezg@um.es**

Interop Testing Results

- **Java client (Rikard) <-> cKUDOS server (Francisco)**
  - Successful single execution in FS mode
  - End up establishing same new Master Secret and Master Salt
  - Two consecutive executions fail due to "Replay Detected" on the server for the first KUDOS request of the second run. This is because new OSCORE Security Contexts are not being initialized with an empty replay window

- **Java server (Rikard) <-> cKUDOS client (Francisco)**
  - Successful single execution in FS mode.
  - End up establishing same new Master Secret and Master Salt

Interop Lessons Learned

- **Specification clarity**
  - Clarify that lexicographic ordering means *shortlex order* specifically
  - State that variable-length (opaque) fields in KUDOS_Expand_Label are length-prefixed
    - This is inherited from TLS and explained when defining the Vector data type
  - Clarify that the empty byte string is encoded as 0x40 (CBOR), even though indicated as 0x

- **Key and Context derivation**
  - When deriving a new OSCORE Security Context, recompute the Sender Key, Recipient Key, **and** Common IV
  - Initialize newly created contexts with an empty replay window and SSN = 0

- **Nonce and replay handling**
  - Ensure AEAD nonce construction for responses uses the sender KID length (S) as the PIV length

Summary and Next Steps

- **Design is in a stable state**
  - No major modifications planned
  - Authors would like to request WGLC
- **KUDOS implementations**
  - Two working interoperated implementations
  - A future step is to also update the implementation in C for Contiki-NG
- **Comments and reviews are welcome!**

Thank you!

Comments/questions?

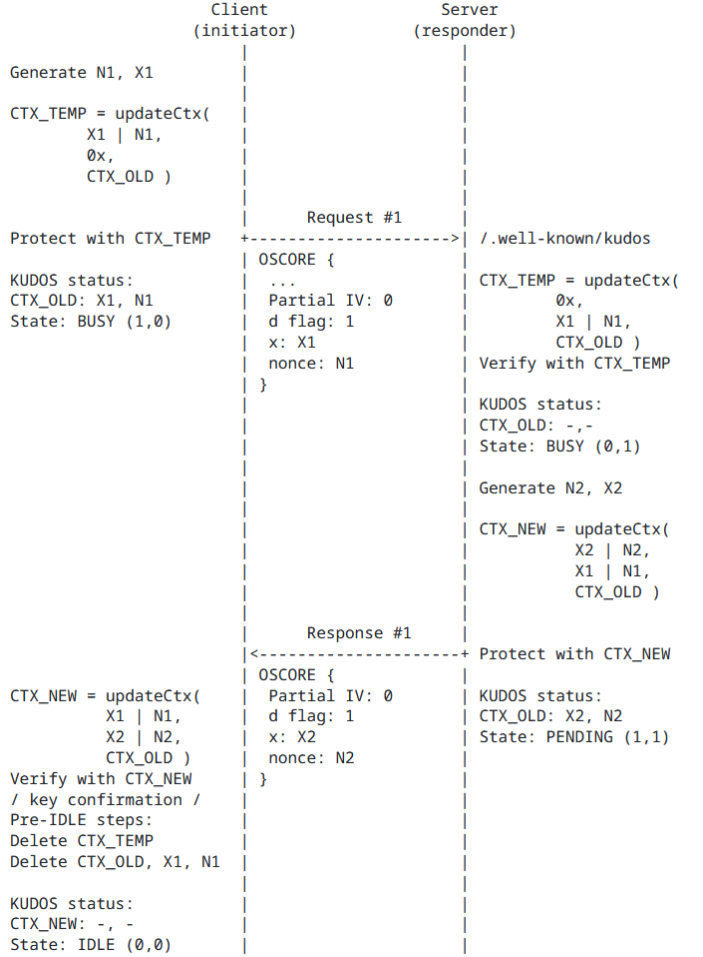
https://github.com/core-wg/oscore-key-update

Backup

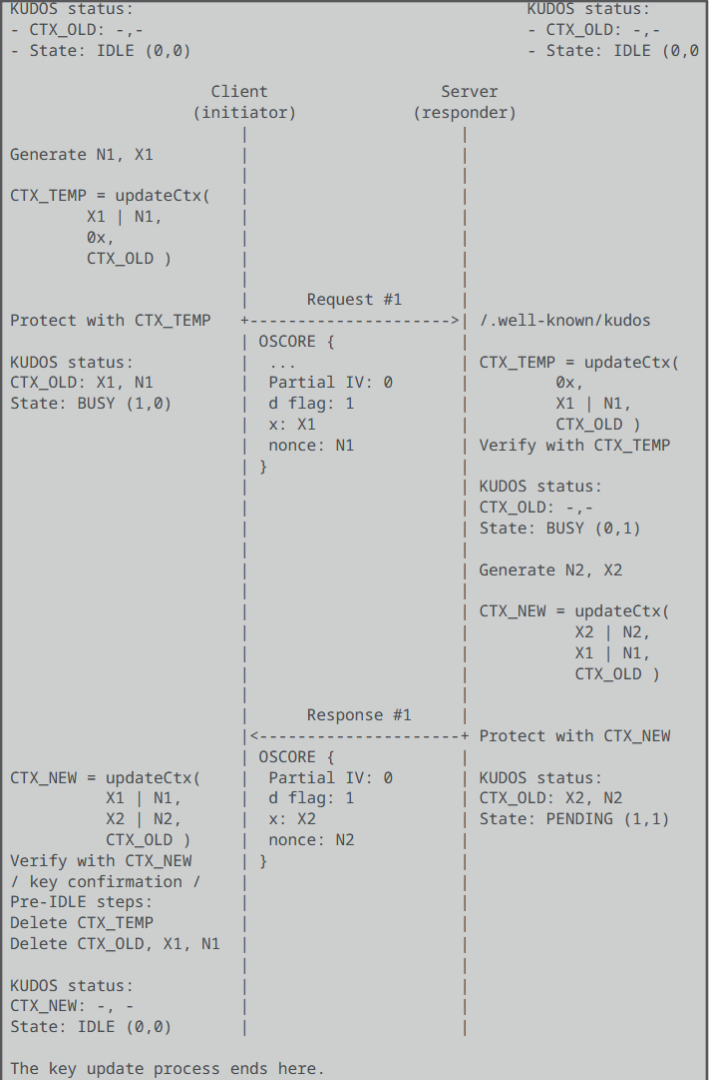
KUDOS Execution Example

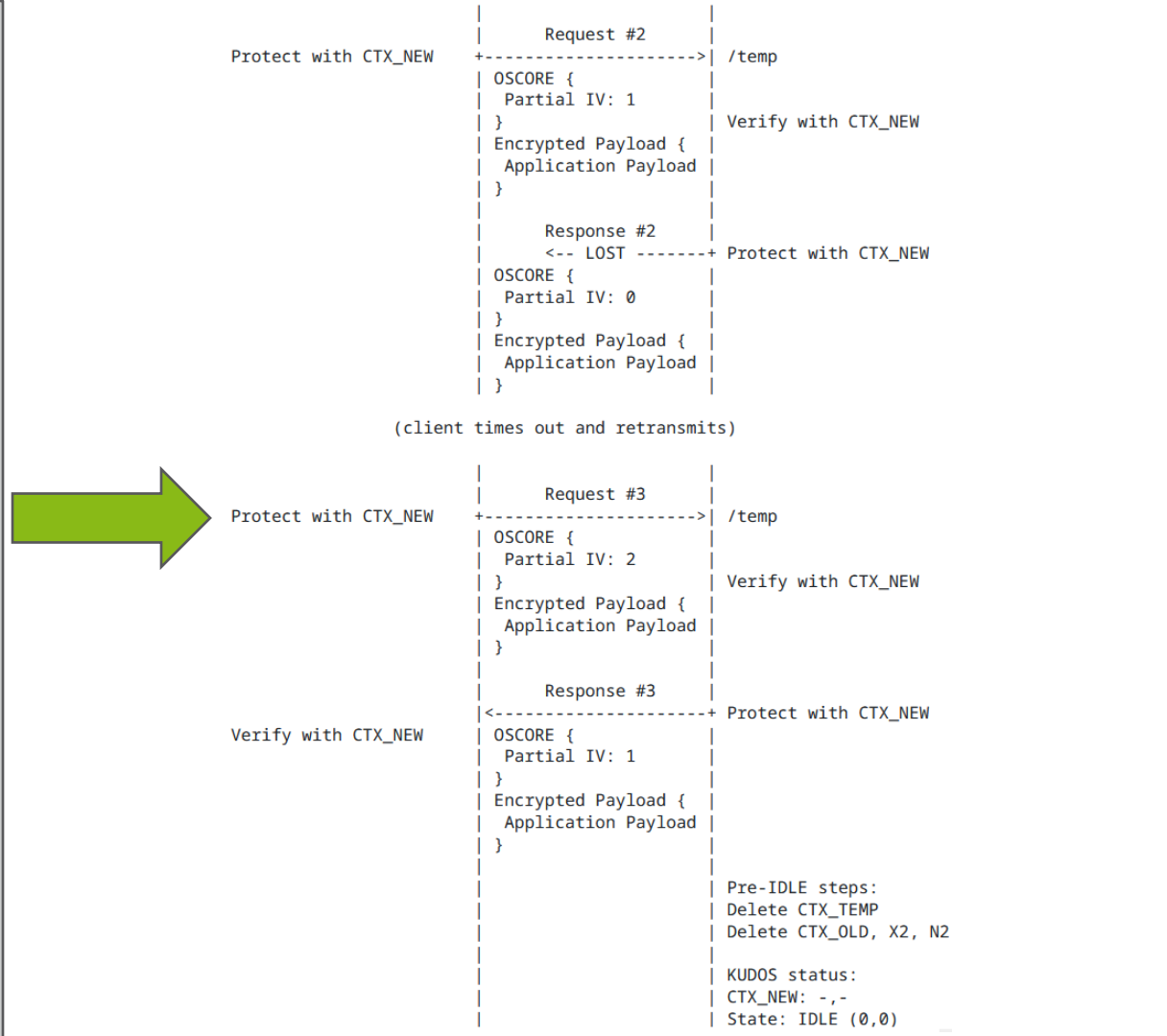
- **Added further message flow example of a KUDOS execution**
  - Successful KUDOS Execution Initiated with a Request Message
  - Successful KUDOS Execution Initiated with a Response Message
  - Successful KUDOS Execution Initiated with a Request Message, with Non-capable Server that has Rebooted
  - Successful KUDOS Execution Initiated with a Request Message, where the Client Executes KUDOS again after the first Execution
  - Successful KUDOS Execution Initiated with a Request Message, where KUDOS Response #1 is Lost
  - Successful KUDOS Execution Completed using two Request Messages
  - Successful KUDOS Execution Initiated with a Request Message, Final Response is Lost

```
KUDOS status:                                   KUDOS status:
- CTX_OLD: -,-                                  - CTX_OLD: -,-
- State: IDLE (0,0)                             - State: IDLE (0,0
```



The key update process ends here.


KUDOS status:
- CTX_OLD: -,-
- State: IDLE (0,0)
Client (initiator)
Server (responder)
Generate N1, X1
CTX_TEMP = updateCtx(
X1 | N1,
0x,
CTX_OLD)
Request #1
Protect with CTX_TEMP
OSCORE {
...
Partial IV: 0
d flag: 1
x: X1
nonce: N1
}
/.well-known/kudos
KUDOS status:
CTX_OLD: X1, N1
State: BUSY (1,0)
CTX_TEMP = updateCtx(
0x,
X1 | N1,
CTX_OLD)
Verify with CTX_TEMP
KUDOS status:
CTX_OLD: -,-
State: BUSY (0,1)
Generate N2, X2
CTX_NEW = updateCtx(
X2 | N2,
X1 | N1,
CTX_OLD)
Response #1
Protect with CTX_NEW
OSCORE {
Partial IV: 0
d flag: 1
x: X2
nonce: N2
}
CTX_NEW = updateCtx(
X1 | N1,
X2 | N2,
CTX_OLD)
Verify with CTX_NEW
/ key confirmation /
KUDOS status:
CTX_OLD: X2, N2
State: PENDING (1,1)
Pre-IDLE steps:
Delete CTX_TEMP
Delete CTX_OLD, X1, N1
KUDOS status:
CTX_NEW: -, -
State: IDLE (0,0)
The key update process ends here.


Request #2
Protect with CTX_NEW
/temp
OSCORE {
Partial IV: 1
}
Verify with CTX_NEW
Encrypted Payload {
Application Payload
}
Response #2
<-- LOST -------
Protect with CTX_NEW
OSCORE {
Partial IV: 0
}
Encrypted Payload {
Application Payload
}
(client times out and retransmits)
Request #3
Protect with CTX_NEW
/temp
OSCORE {
Partial IV: 2
}
Verify with CTX_NEW
Encrypted Payload {
Application Payload
}
Response #3
Protect with CTX_NEW
Verify with CTX_NEW
OSCORE {
Partial IV: 1
}
Encrypted Payload {
Application Payload
}
Pre-IDLE steps:
Delete CTX_TEMP
Delete CTX_OLD, X2, N2
KUDOS status:
CTX_NEW: -,-
State: IDLE (0,0)

Key Usage Limits & ID Update

- **ID Update (draft-ietf-core-oscore-id-update)**
  - **Recap**: Method for updating peers' OSCORE Sender/Recipient IDs. Can be initiated by a client or by a server
  - Submitted new version in January with main change to split overly long sections
  - Next steps
    - Re-consider its design, to be in the same spirit of the new KUDOS design
    - Add more examples including failure cases

- **Key Usage Limits (draft-ietf-core-oscore-key-limits)**
  - **Recap**: OSCORE-specific safe limits for Sender/Recipient Key usage
    - Safe number of encryptions using the Sender Key
    - Safe number of failed decryptions using the Recipient Key
  - Next steps: Align to the document from CFRG as it develops [1]
    - There is ongoing discussion in the CFRG mailing list [2]

[1] https://datatracker.ietf.org/doc/draft-irtf-cfrg-aead-limits/
[2] https://mailarchive.ietf.org/arch/msg/cfrg/CQ6MaMX1t96qxzxJK8cpTPVA46g/